

Declaration of Generative AI Use

FIT3143 Parallel Computing · Lab #1 (Threads & OpenMP) · with summarised prompt records

Student 1 Haruto Iriyama · 34307710 · hiri0001@student.monash.edu

Student 2 Hiew Hia Hao · 34533664 · jhie0006@student.monash.edu

Date 8/19/2026

Generative AI was used during the preparation period for this assessment. It was not used at any point during the presentation or the Q&A session.

All measurements in our slides were produced by compiling and running our own code on our own hardware, and are reproducible from the submitted repository. We understand the code we are submitting and can explain any part of it unaided.

Tool: Claude (Anthropic), via the Claude Code CLI, models Claude Sonnet 5 and Claude Opus 5. Used 17–19 August 2026, in one extended session.

Division of work

Area	AI contribution	Ours
task1.c, task2.c, task3.c	Reviewed our drafts; proposed the <code>estimate_capacity</code> function and the <code>cyclic_thread_func</code> loop as code fragments, with reasoning	Wrote the programs, typed in every change, compiled and verified them, decided what to accept
Tooling (<i>benchmark.sh, summarize.sh, plot.py, optbench.c</i>)	Wrote these scripts	Ran every measurement on our own machine and checked the outputs
Experimental design & analysis	Suggested repetition counts, best-of-N aggregation, the <code>taskset</code> and <code>schedule</code> experiments; helped interpret the numbers	Executed the experiments; checked each explanation against our own data before accepting it
Slides	Drafted structure and content from our measurement data	Reviewed, corrected and completed the deck; will present it unaided

Done without AI: the Docker environment setup, all compilation and execution, the ~350 timing runs, and correctness verification by `diff` against the serial output.

Summarised prompt records

Summarised with the unit's permission rather than reproduced verbatim.

What we asked	Outcome
Explain the unit's Docker setup and how to use VS Code with the container	Explanation of bind mounts and container lifecycle. We set up the environment ourselves.
Should Task 1 replace worst-case allocation with an estimate?	Advised against the common $n / (\ln n - 1)$ form (a lower bound, so it guarantees a reallocation) and proposed $1.2n / \ln n$. Confirmed on request that this is a memory optimisation with no effect on run time — we present it as such.

Compare our shared-array design against an alternative using per-thread buffers	Trade-off analysis. Found a defect in the alternative (count not reset on allocation failure) and noted it used a different <code>is_prime</code> , which would contaminate the comparison. We kept our design.
Is contiguous block partitioning a good approach for Task 2?	No — <code>is_prime(k)</code> costs $\sim\sqrt{k}$, so the highest block finishes last. Proposed cyclic partitioning; we implemented and measured it: +18% to +23% at large n . Our largest optimisation.
Check our work against the specification and rubric	Identified three unmet criteria: near-linear speed-up, "different OpenMP features", and experimental evidence for the Task 1 optimisations. The next three rows are our responses.
Tooling to collect the required measurements	<code>benchmark.sh / summarize.sh</code> . We queried the three-repetition, best-of-N choice; the stated reason (parallel runs vary up to 22%, serial 4%) we confirmed in our own data.
How to evidence the Task 1 optimisations	<code>optbench.c</code> , running three primality tests behind one identical harness. Measured 1,090× at $n = 10^6$; the odd-divisor skip alone gives a consistent 2.0×
Is hard-coding <code>schedule(static, 1)</code> a problem?	The code was fine; what was missing was evidence. Suggested a throwaway copy using <code>schedule(runtime)</code> to compare four schedules, and warned that shipping <code>runtime</code> would silently fall back to the slower default.
Why is our speed-up curve a staircase?	Proposed a <code>taskset</code> pinning test. We measured a 13.3% difference between two and four physical cores, confirming the explanation.
Why 4.7× and not 8×? Is OpenMP really faster?	SMT sharing the integer divider, the $O(n)$ sequential compaction, thread-creation cost. The apparent 20% OpenMP advantage proved to be the partitioning difference, not the API — after fixing Task 2 the two are equal within noise.
Line-by-line explanation of <code>task1.c</code> and <code>task2.c</code>	Pointers, <code>malloc/realloc</code> , <code>sizeof</code> , <code>scanf</code> , <code>struct</code> , the <code>pthread</code> API. We asked precisely because AI is not permitted in the Q&A and we must explain the code unaided.

Two instances where we did not simply accept the output: the AI predicted the contiguous version would cap near 2.8× when we measured 4.08×, and a single run appeared to show `guided` was 9.7% faster until five repetitions showed it was noise. Both are reported as measured.